Terraform Task-2

Task Description:

Create 2 EC2 instances on 2 different regions and install nginx using terraform script.

provider.tf

```
provider "aws" {
region = "us-east-1"
}

provider "aws" {
alias = "west"
region = "us-west-2"
}
```

userdata.sh :

```
#!/bin/bash
sudo apt update -y
sudo apt install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx

echo "<h1>Terraform Nginx Server</h1>" > /var/www/html/index.html
```

main.tf:

```
# Fetch latest Ubuntu AMI in us-east-1
data "aws_ami" "ubuntu_east" {
  most_recent = true

  filter {
```

```
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-focal-*-amd64-server-*"]
  }

  owners = ["099720109477"]
}

# Fetch latest Ubuntu AMI in us-west-2
data "aws_ami" "ubuntu_west" {
  provider     = aws.west
  most_recent  = true

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-focal-*-amd64-server-*"]
  }

  owners = ["099720109477"]
}

# ---------- EC2 Instances ----------

resource "aws_instance" "east_server" {
  ami           = data.aws_ami.ubuntu_east.id
  instance_type = "t3.micro"           # updated
  user_data     = file("userdata.sh")

  tags = {
    Name = "Nginx-East-Server"
  }
}

resource "aws_instance" "west_server" {
  provider      = aws.west
  ami           = data.aws_ami.ubuntu_west.id
  instance_type = "t3.micro"           # updated
```

```
  user_data     = file("userdata.sh")

  tags = {
    Name = "Nginx-West-Server"
  }
}
```

[outputs.tf](#):

```
output "east_instance_ip" {
  value = aws_instance.east_server.public_ip
}

output "west_instance_ip" {
  value = aws_instance.west_server.public_ip
}
```

```
hp@LAPTOP-M939AS8F MINGW64 ~/terraform-nginx-ec2
$ ls -ltr
total 27
-rw-r--r-- 1 hp 197121   103 Mar 10 19:18 provider.tf
-rwxr-xr-x 1 hp 197121   179 Mar 10 19:19 userdata.sh*
-rw-r--r-- 1 hp 197121   151 Mar 10 19:20 outputs.tf
-rw-r--r-- 1 hp 197121  1004 Mar 10 19:23 main.tf
-rw-r--r-- 1 hp 197121 18287 Mar 10 19:49 terraform.tfstate
```

Terraform init:

```
$ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.35.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

hp@LAPTOP-M939AS8F MINGW64 ~/terraform-nginx-ec2
```

Terraform plan:

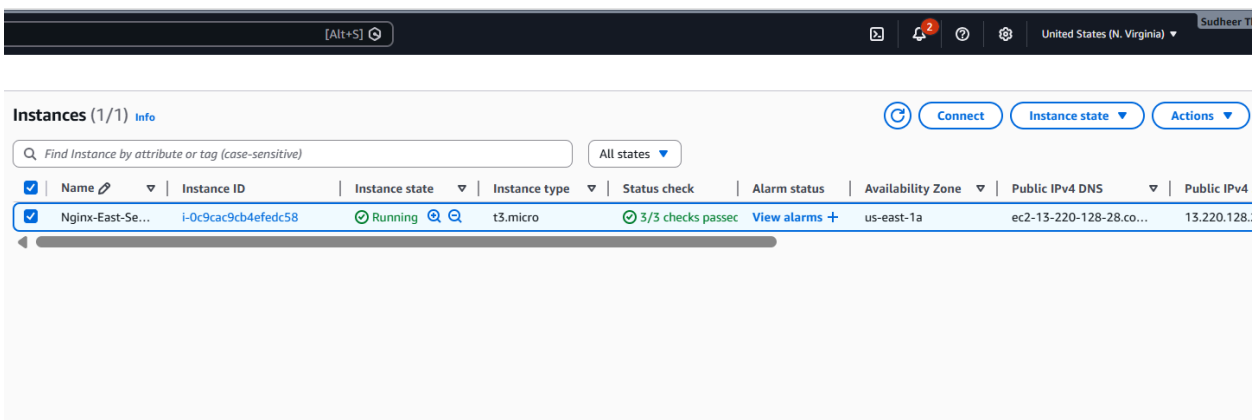
```
$ terraform plan
data.aws_ami.ubuntu_east: Reading...
data.aws_ami.ubuntu_west: Reading...
data.aws_ami.ubuntu_east: Read complete after 2s [id=ami-0fb0b230890ccd1e6]
data.aws_ami.ubuntu_west: Read complete after 2s [id=ami-0a15226b1f7f23580]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.east_server will be created
  + resource "aws_instance" "east_server" {
      + ami                                  = "ami-0fb0b230890ccd1e6"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
```

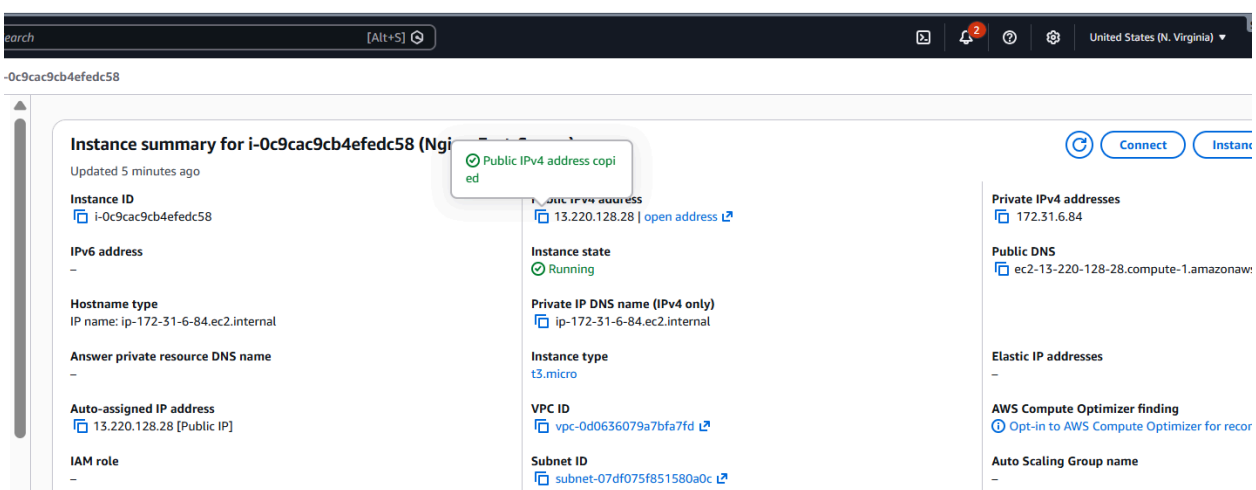
Terraform plan:

```
$ terraform apply
data.aws_ami.ubuntu_east: Reading...
data.aws_ami.ubuntu_west: Reading...
data.aws_ami.ubuntu_east: Read complete after 2s [id=ami-0fb0b230890ccd1e6]
data.aws_ami.ubuntu_west: Read complete after 2s [id=ami-0a15226b1f7f23580]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.east_server will be created
  + resource "aws_instance" "east_server" {
      + ami                                  = "ami-0fb0b230890ccd1e6"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
```

```
      + secondary_network_interface (known after apply)
    }

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + east_instance_ip = (known after apply)
  + west_instance_ip = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.east_server: Creating...
aws_instance.west_server: Creating...
aws_instance.east_server: Still creating... [00m10s elapsed]
aws_instance.west_server: Still creating... [00m10s elapsed]
aws_instance.east_server: Creation complete after 18s [id=i-0c9cac9cb4efedc58]
aws_instance.west_server: Creation complete after 18s [id=i-0542beee238808e9a]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

east_instance_ip = "13.220.128.28"
west_instance_ip = "35.86.253.120"
```

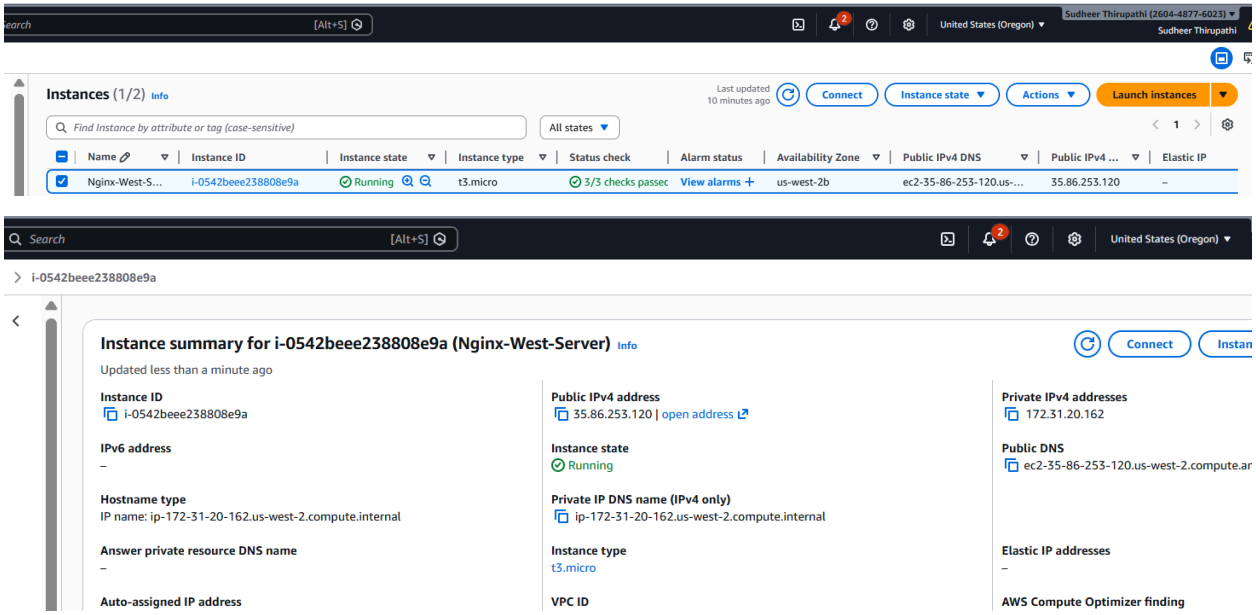
Us-east-1:

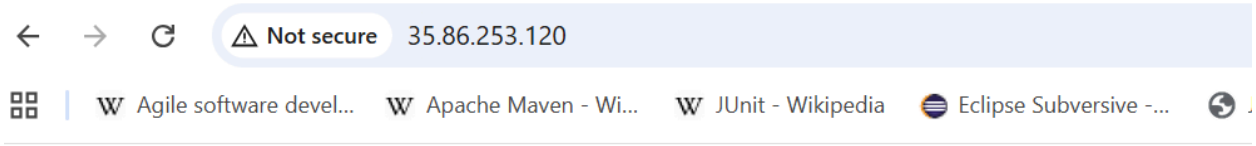




Terraform Nginx Server

Us-west-2:





Terraform Nginx Server